Introduction to Software Engineering Project- HW 2

Gameplay Example:

The inputs I entered in order:

3 -> 1 -> 1 -> 1 -> 1 -> 1 -> 1 -> 1

The outputs and the game situation in each player decision:

```
First fight:
Player HP: 100 Player Resource: 5 a HP: 30
3  (Special)
Player HP: 90 Player Resource: 2 a HP: 15
1  (Attack)
Player HP: 80 Player Resource: 2 a HP: 5
1  (Attack)
Next round, second fight:
Player HP: 80 Player Resource: 2 b HP: 45
1  (Attack)
Player HP: 70 Player Resource: 2 b HP: 35
1  (Attack)
Player HP: 60 Player Resource: 2 b HP: 25
1  (Attack)
Player HP: 50 Player Resource: 2 b HP: 15
1  (Attack)
Player HP: 40 Player Resource: 2 b HP: 5
1  (Attack)
You win!
```

I removed the repeated action menu from this example so the changes in the HP and resource are easier to read. The values and the win message are from my actual run. The input number is written next to the action to make the example clearer.

Explaining the outputs:

In the first fight I used Special and then Attack twice. The first enemy died after the third action, so it did not hit me again. I started the next fight with the same 80 HP and 2 resources, but with a new enemy that had 45 HP. After defeating this enemy the game printed You win!

Tests and results:

For each test I used a new run or a separate small test setup. These are the results I got with my Week 2 code:

test	input / setup	expected	Actual
T1 / W02-OBJ-01	Two enemies (30 and 45 HP). Damage A for 10.	A = 20, B = 45	A = 20, B = 45
T2 / W02-ENC-01	3 -> 1 -> 1	Second fight: HP 80, res 2	HP 80, res 2
T3 / W02-ENC-02	Win fight 1	New enemy: 45 HP	b HP = 45
T4 / W02-ENC-03	Test setup: player HP 80, res 2, defence true. Start battle; Attack.	Next hit takes 10 HP	HP 80 -> 70
T5 / W01-INV-01	3 -> 3	Second Special rejected, no enemy hit	HP 90, enemy 15, res 2 unchanged
T6 / W01-END	3 -> 1 -> 1	No hit after enemy dies	Second fight starts at HP 80
T7 / TERM	2 repeated 20 times	Player loses; no second battle	You lose! No second fight

T1 and T4 were checked using a separate test setup, because the normal game does not let me choose these exact starting conditions. I did not change the submitted Main.java to run these checks.

A few outputs from the tests:

```
T1: A HP=20; B HP=45
T4: Fixture before Encounter 2: HP=80, resource=2, defense=true
    Player HP: 80 Player Resource: 2 b HP: 45
    Player HP: 70 Player Resource: 2 b HP: 35
T5: Player HP: 90 Player Resource: 2 a HP: 15
    Player HP: 90 Player Resource: 2 a HP: 15
T7: You lose!
```

Edge cases:

I also tested some values at the limits, to check that resource cannot become greater than 10 and HP does not become negative.

test	expected	Actual
Special with 2 resource	Action rejected	false; resource 2, enemy HP 45
Special with 3 resource	Resource becomes 0	true; resource 0, enemy HP 30
Recover with 9 resource	Resource becomes 10	true; resource 10
Recover with 10 resource	Action rejected	false; resource 10
Attack enemy at 5 HP	Enemy HP becomes 0	Enemy HP 0
Invalid input: x	No state change or counterattack	Player 100, enemy 30, resource 5

Design Note:

The main design decision I made this week was to use three classes: Main, Battle and Combatant. In Week 1 I had separate classes for the player and the enemy, but I noticed that both needed almost the same things, like HP and taking damage. So I decided to use Combatant for both of them. It stores the name, HP, resource and defence of each character. I kept these variables private because I did not want other classes to change them directly and accidentally break the game rules. For example, the special method checks that there are at least 3 resources, and recover does not let the resource go above 10. Battle is responsible for reading the action, letting the player play, checking if the enemy is still alive and deciding if the enemy can attack back. I did not put the Scanner or the order of the encounters in Combatant, because they are not properties of one character. Main creates the player once and uses it in both encounters. This is important because the variable points to the same object, so its HP and resources do not reset after the first fight. Each fight creates a new enemy object, so changing the first enemy does not affect the second one. The temporary defence is cleared before the next battle.

Engineering quality and AI task:

Prompt summary - I asked the AI to suggest a simple structure for the two-encounter game and explain the responsibilities of Main, Battle and Combatant. I did not ask it to write the whole game.

Suggestion I used:

The AI suggested keeping the character state inside Combatant and keeping the turn logic in Battle. I found this useful because it makes it easier to understand which class is responsible for each part of the program.

Suggestion I rejected:

One idea was using inheritance and different subclasses for the player and enemy. I did not use this because both characters have the same basic data for this assignment, and I thought it would make the code more complicated than necessary.

Test I used:

I tested the same player object in two fights by entering 3 -> 1 -> 1 in the first fight. The second fight began with 80 HP and 2 resource, and with a new enemy at 45 HP. This showed me that the player keeps its state while each enemy is independent.

Conclusion:

I tested the two fights, the main boundary cases and the cases where the player or an enemy dies. The program worked as expected in these tests, so I kept my current class structure.